LEARN · VERIFY — OFFLINE STUDY BOOKLET

# Transactions

The promise that lets you pretend crashes and concurrency don't exist — what it actually says, every discount you are sold on it, and what each discount lets through.



Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

# How to use this booklet

## The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

> **THE TRAP THIS SUBJECT SETS, SPECIFICALLY**
>
> Almost every term here has a look-alike, and several have the *same name* in different databases. “Repeatable read” means three different things depending on the vendor; “serializable” sometimes means snapshot isolation; “atomic” means one thing in ACID and a different thing in a thread. Whenever you meet a term in these pages, say which of its neighbours it is *not*. That habit is most of the competency.

## The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

## What's in here

SECTION 1

# Learning goal & roadmap

**LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS**

Given a real design or incident (“two people booked the same room”, “the balance went negative and nobody noticed”, “the totals in this report don't add up”, “half the cluster is blocked on one stuck transaction”), you can:

1. **Name the anomaly** — dirty read, dirty write, read skew, lost update, write skew, phantom — and say which isolation level would have prevented it.
2. **Locate the decision**: what did the transaction *read*, what did it *decide*, and what did it *write*? Nearly every bug in this competency is a decision made on a premise that stopped being true.
3. **Prescribe the fix** at the right layer — an atomic operation, an explicit lock, a uniqueness constraint, a stronger isolation level, or a redesign around idempotence — and name the near-miss you rejected.
4. **State the price.** Every level above the one you are on costs throughput, latency predictability, or aborts you now have to retry.

## The core model in one paragraph

A transaction is a bargain. You group several reads and writes and say “treat these as one”; in exchange the database promises that a fault mid-way leaves no trace and that concurrent work cannot show you a half-finished state. That promise, honoured in full, is called **serializable**: the result is as if the transactions had run one at a time. Full serializability has a real cost, so almost every database ships you something cheaper by default and still calls it a transaction. Everything difficult about this subject lives in the gap. The weaker levels each prevent a specific list of anomalies and permit the rest — and the ones they permit cannot be found by testing, because they need unlucky timing to appear. Part 1 is the promise. Part 2 is what the two common discounts do buy you. Part 3 is what they leave on the floor, and the three ways to buy the promise back. Part 4 is what happens to all of it when the transaction has to span more than one machine.

**TENTH-GRADER VERSION — THE WHOLE THING AT ONCE**

- A transaction is a way of saying “all of this, or none of it” — so if something breaks halfway, you can just try again.
- It's also a way of saying “don't let anyone else see my work until I'm finished.”
- Doing that perfectly is slow, so databases sell you a cheaper version and don't always tell you which one.
- The cheaper versions all have the same shape of bug: you look, you decide based on what you saw, and by the time you write, what you saw isn't true any more.
- And when the transaction has to cover two machines, someone has to be in charge of deciding it happened — which is fine right up until that someone dies mid-sentence.

## The four parts, and why they're in this order

| PART | WHAT IT COVERS | WHY HERE |
|---|---|---|
| **1 What a transaction promises** | Atomicity as abortability; consistency as the application's job; isolation as serializability; durability and its limits. Single-object versus multi-object operations, and why multi-object ones are needed at all. Aborts, retries, and the five ways a retry can hurt you. | You cannot judge a discount without knowing the full price. Every later part is a subtraction from this. |

| | | |
|---|---|---|
| **2** **The weak levels, and what they stop** | Read committed: no dirty reads, no dirty writes, and how each is implemented. Read uncommitted. Read skew, and snapshot isolation as the answer. Multiversion concurrency control, visibility rules, and the naming confusion around “repeatable read”. | These two levels cover the overwhelming majority of real deployments. Knowing exactly what they guarantee is the practical core. |
| **3** **What they don't stop** | Lost updates and the five ways to prevent them. Write skew as the generalisation. Phantoms, and why an explicit lock has nothing to hold on to. Materialising conflicts. Then the three implementations of serializability: actual serial execution, two-phase locking, and serializable snapshot isolation. | The anomalies here are the ones that reach production, because no weak level catches them and no test reliably reproduces them. |
| **4** **When the transaction spans machines** | The atomic commitment problem. Two-phase commit, its two points of no return, in-doubt participants and the locks they hold. Coordinator failure. XA and why it disappoints. Database-internal distributed transactions. Exactly-once processing without any of it. | Atomicity on one node comes down to one disk writing one record. Across nodes it becomes a protocol — and the protocol has a failure mode that stops your application. |

> *“We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions.”*

1

PART 1 OF 4

# What a transaction promises

Four letters, three of which do not mean what the word suggests.

### TENTH-GRADER VERSION

- A transaction is a bundle of changes with a rule attached: either all of them happen, or none of them do.
- That rule is worth more than it sounds. It means that when something goes wrong, you know *nothing* happened — so it is safe to just do it again.
- There is a second rule too: while you're in the middle of your bundle, nobody else gets to see the half-done version.
- The four famous letters are A, C, I and D. Three of them are misleading names, and one of them isn't really the database's job at all.

### THE EVERYDAY VERSION

Moving house. You are carrying a stack of plates from the van to the kitchen. **Atomicity** is the rule that if you drop the stack half-way, you don't end up with three plates in the kitchen and four on the drive — you end up back at the van, able to try again knowing exactly where you stand. **Isolation** is the rule that nobody walks into the kitchen mid-trip and counts the plates. **Durability** is the rule that once the plates are on the shelf they stay there. And **consistency** — "there should be eight plates in this house" — is a rule you hold in your head. The van doesn't know it, and won't stop you breaking it.

## The terms, each with its look-alike

| TERM | PLAIN DEFINITION | CONCRETE EXAMPLE | BOUNDARY — WHAT IT GETS CONFUSED WITH |
|---|---|---|---|
| **Transaction** | Several reads and writes grouped into one logical unit that either **commits** or **aborts**. | In a relational database, everything between `BEGIN TRANSACTION` and `COMMIT` on one client connection. | vs **a multi-object API call** such as a key-value store's multi-put, which may succeed for some keys and fail for others. Grouping operations is not the same as transaction semantics; without a way to identify the group and undo it, you have neither. |
| **Atomicity (the A)** | If a fault occurs after some of the writes, the transaction is aborted and every write it made so far is discarded. | An error updating the unread counter rolls back the email insertion too, so the mailbox and the counter cannot disagree. | vs **atomicity in multithreaded programming**, where it means no other thread can see a half-finished operation. That sense is covered by the letter I here. ACID atomicity is **not about concurrency at all** — **abortability** would have been the better word. |

| | | | |
|---|---|---|---|
| **Consistency (the C)** | An application-specific invariant that must be true before and after — credits and debits balance, a balance never goes negative. | Foreign-key, uniqueness and check constraints declared in the schema; more complex invariants sometimes via triggers or materialized views. | vs the **four other things the word means** — replica consistency, a consistent snapshot, consistent hashing, and linearizability in the CAP theorem. The C is also the odd letter out: it depends on how the application uses the database, not on the database alone. Invariants you never declared cannot be enforced. |
| **Isolation (the I)** | Concurrently executing transactions cannot step on each other. Formalised as **serializability**: each transaction may pretend it is the only one running. | Two clients both incrementing a counter from 42 end at 44, not 43. | vs **what you actually get**. Serializability costs performance, so most databases default to something weaker and still call it isolation — one popular system's isolation level named “serializable” in fact implements snapshot isolation. The word on the tin is not the guarantee. |
| **Durability (the D)** | Once committed, the data will not be forgotten, even after a crash. | On one node: `fsync` to nonvolatile storage, a write-ahead log for crash recovery, checksums to detect incomplete entries. In a replicated system: copied to some number of nodes before the commit is reported. | vs **a guarantee**. There is no perfect durability, only stacked risk reduction. Disks and their firmware have bugs, `fsync` is hard to use correctly, a correlated fault can take out every replica at once, and asynchronous replication can lose recent writes at failover. Write to disk *and* replicate *and* back up. |
| **Single-object operation** | Atomicity and isolation applied to one row, document or key — implemented with a crash-recovery log and a per-object lock. | Writing a 20 kB JSON document: no reader sees half of it, a power cut doesn't splice old and new together, an interrupted send doesn't store an unparseable fragment. Also atomic increments and conditional writes. | vs **a transaction**. These are useful and can prevent lost updates on one object, but they give **no guarantee across multiple objects**. A store offering linearizable reads and conditional writes on single objects has not given you transactions. |
| **Multi-object transaction** | One unit spanning several rows, documents or index entries. | Needed when a foreign key must stay valid, when denormalized data must stay in sync with its source, and whenever a **secondary index** must be updated with the row — the index is a separate object. | vs **doing it without transactions**, which is possible: error handling becomes much more complicated without atomicity, and the lack of isolation produces the concurrency bugs the rest of this booklet is about. |
| **Abort · rollback · retry** | Abandoning a transaction and undoing its writes, so the application can safely try again. | A deadlock, an isolation violation, a transient network interruption, a failover. | vs **best-effort systems**, whose philosophy is “do as much as possible and don't undo what's done” — leaving error recovery to the application. Datastores with leaderless replication typically work this way. |

## Retrying is the point of aborting — and it can still bite you

Rolling back exists so that a retry is safe, which makes it the single best error-handling mechanism available. It is worth noticing that popular object-relational mapping frameworks do **not** retry aborted transactions: the error becomes an exception, the user's input is thrown away, and they get an error message. That wastes the whole mechanism. But a retry is not free either:

- **It may have already succeeded.** If the commit worked and the acknowledgment was lost, retrying performs it twice — unless you have an application-level deduplication mechanism.
- **It can make an overload worse.** If the abort was caused by load or contention, retrying adds to the load. Limit the number of retries, use exponential backoff, and treat overload errors differently from other errors.

- **It is pointless after a permanent error.** Retry after a deadlock, an isolation violation, a temporary network interruption or a failover. Do not retry a constraint violation.
- **Side effects outside the database still happen.** If the transaction sent an email, aborting doesn't unsend it, and every retry sends another.
- **The client can die mid-retry**, in which case the data it was trying to write is simply lost.

> **CONCEPT BOUNDARY — WHAT "ACID COMPLIANT" DOES *NOT* TELL YOU**
>
> **It does not** tell you which isolation level you get, because implementations of ACID differ from one database to the next and there is a great deal of ambiguity in the meaning of isolation. As a claim, it has become largely a marketing term.
> **It also isn't** usefully contrasted with the label sometimes applied to systems that don't meet the criteria — "basically available, soft state, eventual consistency" — which is vaguer still. The only sensible reading of that label is *not ACID*, which can mean almost anything.
> **What to ask instead:** which isolation level is the default, what is it actually called in this product, and which anomalies does that level permit?

## Anchor sketch — the four letters, unpacked

**Legend: bold = a named node** **reversed = the node that matters most** **Q = cover the page and answer this**

```
   a transaction = many reads and writes, ONE unit
            it COMMITS or it ABORTS. never half.
                        |
ATOMICITY   a fault part-way through discards every
            write made so far. nothing to do with
            concurrency.
            |
            +--> the honest name is ABORTABILITY, and
                 that is exactly what makes a RETRY safe
            |
CONSISTENCY the invariant is YOURS. the database can
            enforce only what you declared as a
            constraint.
            |
            +--> the odd letter out: a property of how
                 the application uses the database
            |
ISOLATION   concurrent transactions cannot see each
            other's half-done work.
            |
            +--> formalised as SERIALIZABLE -- and this
                 is the letter everyone quietly discounts
            |
DURABILITY  a committed write survives a crash.
            fsync . write-ahead log . checksums . replicas
            |
            +--> no perfect durability exists. stack the
                 risk reductions; do not trust one.
```

**Q** Which letter covers "another client sees my half-finished work", and which covers "a crash left my work half-finished"?

**Q** Why is C the odd one out, and what happens to an invariant you never declared?

**Q** Name three reasons a retry after an abort can still hurt you.

## Trade-offs

| DECISION | BUYS YOU | COSTS YOU | CHOOSE IT WHEN |
|---|---|---|---|
| Use transactions at all | A large class of errors collapses into “abort and try again”. You get to ignore partial failure and most concurrency. | Throughput, and a dependency on getting the isolation level right. Not every application needs them. | Access patterns are more complex than reading and writing single records — which is most of the time. |
| Single-object atomic operations instead | Concurrency-safe increments and conditional writes with no transaction and no read-modify-write cycle in your code. | Nothing across objects. And not every change can be expressed as one — arbitrary text editing, for instance. | The invariant genuinely lives inside one object. |
| Declaring invariants as constraints | The database enforces them for you, including against code you didn't write. | Complex invariants are hard or impossible to express this way, and constraints spanning several objects usually aren't supported at all. | Always, for the ones that fit: uniqueness, foreign keys, value checks. |
| Durability by disk | Survives a crash of the process and the machine. | If the machine dies, the data is intact but **inaccessible** until you fix it or move the disk. | Alongside replication, never instead of it. |
| Durability by replication | The system stays *available* through a machine failure. | A correlated fault can take out every replica at once, and asynchronous replication can lose recent writes at failover. | Alongside disk writes and backups. |
| Automatic retry of aborted transactions | Simple, effective error handling — the reason rollback exists. | Double execution if an acknowledgment was lost; a worse outage under contention; nothing at all for side effects outside the database. | For transient errors, with bounded retries and backoff. |

## Say it so you understand it → say it like an expert

| SAY IT THIS WAY TO *UNDERSTAND* IT | SAY IT THIS WAY TO SOUND LIKE AN *EXPERT* |
|---|---|
| “It's all-or-nothing.” | “That's ACID atomicity, which is really abortability — the guarantee that lets us retry safely. It says nothing about concurrency; that's the I.” |
| “The database keeps our data consistent.” | “It enforces the invariants we declared as constraints. The ones we didn't declare are our problem — the C in ACID is a property of how we use it.” |
| “We're on an ACID database, so we're fine.” | “Which isolation level, and what's it called here? Most defaults are well below serializable, and the naming is inconsistent between products.” |
| “The write is committed, so it's safe.” | “It's durable to the extent our stack is. Disk plus replication plus backups — perfect durability doesn't exist, and a correlated fault can take every replica.” |
| “We do a multi-put, so it's atomic.” | “A multi-object API isn't transaction semantics. That call can succeed for some keys and fail for others and leave us partially updated.” |
| “It failed, so we'll just run it again.” | “Only if the error was transient and there are no external side effects — and we need deduplication in case the commit actually succeeded and the ack was lost.” |

**PART 1 · QUESTION 1**

A team stores each order as a single document and also maintains a separate `order_totals` summary row per customer, plus a secondary index on order status. They have no multi-object transactions — the store offers atomic single-document updates and a conditional write, which they describe as “atomic, so we're covered”. Support reports two recurring issues: totals that don't match the sum of a customer's orders, and orders that appear when filtering by status but not when listed directly.

**(a)** Name the two guarantees they believe they have and say precisely what each one does and does not cover. **(b)** Explain the mechanism behind *each* of the two reported symptoms. **(c)** The team proposes retrying any failed write until it succeeds. Give two distinct reasons this does not fix the problem, one of which must not depend on the network.

**PART 1 · QUESTION 2**

A payments service runs on a database advertised as ACID compliant. During an incident review, an engineer says: “the database guarantees consistency, so a negative balance is impossible”. A second engineer says: “and it's durable, so once we return 200 to the client the money has definitely moved”. Neither statement is safe as stated.

**(a)** Correct the first statement, naming what the database will and will not enforce and what the team must do about it. **(b)** Correct the second, giving two distinct ways a committed write can still be lost. **(c)** The service sends a confirmation email inside the transaction and retries the whole transaction on any error. Name the specific hazard and say what has to change.

## 2×2 — how many objects, how many clients

COLUMNS: IS MORE THAN ONE *CLIENT* INVOLVED? →

| | One client at a time | Several clients concurrently |
|---|---|---|
| **ONE object** | **Already solved for you**<br>Storage engines almost universally give atomicity and isolation on a single object: a crash-recovery log, plus a lock so only one thread touches it.<br>*Move:* nothing. No reader ever sees half a document, and a power cut doesn't splice old and new values together. | **Atomic operations and conditional writes**<br>An increment done by the database, or a write that applies only if the value hasn't changed since you read it.<br>*Move:* use them, and stop writing read-modify-write cycles in application code. This prevents lost updates **on that object** — and gives you nothing across objects. |
| **MANY objects** | **Atomicity is what you need**<br>A fault part-way through leaves the mailbox and the counter disagreeing, or a foreign key dangling.<br>*Move:* a multi-object transaction, so an error rolls the whole thing back and you can retry knowing nothing took effect. Note that a secondary index makes almost every write multi-object whether you meant it or not. | **Where this whole booklet lives**<br>Several clients, several objects each. Every anomaly in parts 2 and 3 needs exactly this shape.<br>*Move:* you need A **and** I together, and you must know which grade of I you are actually getting. Assume the default is well below serializable until you have checked the product's own name for it. |

**Read it as:** going down is what atomicity is for; going right is what isolation is for. People routinely reach for a bottom-left tool in a bottom-right situation — “we use an atomic increment, so we're safe” — and the tool is real, it just doesn't cover the axis they're on.

### CARRY-OUT RULE FROM THIS PART

**Say which letter you are relying on.** A crash mid-way is the A; another client seeing half your work is the I; an invariant nobody declared is nobody's job. Most arguments about “is this safe?” are two people relying on different letters without saying so.

2

PART 2 OF 4

# The weak levels, and what they stop

Read committed, snapshot isolation, and the machinery underneath both.

### TENTH-GRADER VERSION

- The safest setting is slow, so databases offer cheaper settings that block *some* problems.
- The cheapest useful one says: you'll never see someone's unfinished work, and you'll never scribble over it.
- But it lets you look at different parts of the database at different moments — so you can add up numbers that were never all true at once.
- The next one up fixes that by giving your whole transaction a photograph of the database taken when it started.
- The photograph is made by keeping several old copies of every row and hiding the ones that are too new for you.

### THE EVERYDAY VERSION

You're auditing a shop's stock by walking round with a clipboard. **Read committed** is the rule that you only count items that have actually been put on the shelf, never ones still in someone's arms. Fine — but you count aisle 1 at ten past, aisle 7 at half past, and in between a box was moved from 7 to 1. You counted it twice, and every single number you wrote down was true when you wrote it. **Snapshot isolation** is being handed a photograph of the whole shop taken the moment you walked in. You count from the photo. Nothing you count moves, and other people keep shopping the whole time.

## The terms, each with its look-alike

| TERM | PLAIN DEFINITION | CONCRETE EXAMPLE | BOUNDARY — WHAT IT GETS CONFUSED WITH |
|---|---|---|---|
| **Dirty read** | Seeing data written by a transaction that has not yet committed. | User 1 sets x = 3 and hasn't committed; user 2 reads x and gets 3. | vs **read skew**, where everything you read *was* committed. Preventing dirty reads matters for two reasons: you'd otherwise see some of a transaction's rows and not others, and you'd be able to read data that is later rolled back — which forces **cascading aborts** of everyone who read it. |
| **Dirty write** | Overwriting a value that another transaction has written but not yet committed. | Two people buying the same car: one wins the update to the listing, the other wins the update to the invoice, so the car goes to one and the bill to the other. | vs a **lost update**, where the earlier write *had* committed — so it isn't a dirty write, it's still wrong for a different reason. Almost all transaction implementations prevent dirty writes; far fewer prevent lost updates. |
| **Read committed** | The basic isolation level: **no dirty reads** and **no dirty writes**. The default in several major databases. | Dirty writes are prevented with **row-level locks** held until commit or abort. Dirty reads are prevented by remembering both the old committed value and the new uncommitted one, and serving readers the old one. | vs preventing dirty reads **with read locks**, which is possible and is what some databases do, but works badly: one long-running write transaction makes read-only transactions queue behind it, so a slowdown in one part of the application shows up in a completely unrelated part. |

| | | | |
|---|---|---|---|
| **Read uncommitted** | Weaker still: prevents dirty writes but **not** dirty reads. | Returns the latest written value immediately, even from an uncommitted transaction. | It's cheaper because the database needn't store two versions of the row. It *reduces the probability* of lost updates without preventing them — which is not a guarantee you can build on. |
| **Read skew** (nonrepeatable read) | Seeing different parts of the database at different points in time. | $500 in each of two accounts. A transfer of $100 runs while you look; you see one account before the money arrives ($500) and the other after it left ($400), so $1,000 looks like $900. | vs a **dirty read** — nothing here was uncommitted. Both balances were genuinely committed at the moment they were read. This is why read skew is considered acceptable under read committed. Note the word **skew** is overloaded; elsewhere it means an unbalanced workload, here it means a timing anomaly. |
| **Snapshot isolation** | Each transaction reads from a **consistent snapshot**: everything committed at the moment it started, and nothing later. | Essential for backups, which take hours and would otherwise mix old and new parts, and for analytical queries and integrity checks that scan large parts of the database. | vs **serializable**. It is not serializable, however it is labelled: it permits write skew and, in some implementations, lost updates. Also vs the **SQL standard's repeatable read** — the standard predates snapshot isolation and defines something that looks superficially similar. |
| **Multiversion concurrency control (MVCC)** | Keeping several committed versions of each row side by side, so different transactions can see the database at different points in time. | Each row carries an `inserted_by` and a `deleted_by` transaction id. An update is internally a delete plus an insert. A garbage collector removes versions no transaction can still see. | vs the **two-version** trick used for read committed alone (old committed value plus new uncommitted one). MVCC is the generalisation of it. Its defining performance property: **readers never block writers, and writers never block readers** — write locks still prevent dirty writes, but reads take no locks at all. |
| **Repeatable read** | A name, not a guarantee. | In one database it means snapshot isolation; in another it means an MVCC implementation weaker than snapshot isolation; in a third it means serializability. | The SQL standard's definition is ambiguous and imprecise, several databases implement things they all call repeatable read with big differences between them, and most don't satisfy the formal research definition. Treat the phrase as a prompt to go and read the specific product's documentation. |

## How a consistent snapshot is actually decided

Every transaction gets a unique, always-increasing transaction id when it starts. Every row it writes is tagged with that id. When a transaction reads, the database decides which row versions are visible to it:

1. At the start of the transaction, the database lists all **other transactions in progress** at that moment. Writes by those transactions are ignored **even if they subsequently commit**.
2. Writes by transactions with a **later id** are ignored, regardless of whether they have committed.
3. Writes by **aborted** transactions are ignored, whenever the abort happened. This is convenient: an aborting transaction's rows don't need removing immediately, because the visibility rule already filters them out and the garbage collector can clean up later.
4. Everything else is visible.

Put the other way round, a row is visible if the transaction that *inserted* it had already committed when the reader started, and it is either not marked for deletion or the transaction that requested the deletion had not committed when the reader

started. Because values are never updated in place — a new version is inserted instead — the snapshot costs only a small overhead, and a long-running transaction can keep reading values everyone else considers long gone.

#### TWO THINGS THAT FOLLOW FROM "NEVER UPDATE IN PLACE"

**Indexes need care.** The common approach is for each index entry to point at one version of a row, with versions chained together; a query using the index walks the chain for a version that is both visible and matching. When the garbage collector removes invisible versions, the corresponding index entries go too.

**There is a second design.** Some databases use an immutable, copy-on-write B-tree: a write copies each modified page and every parent up to the root, so every write transaction creates a **new tree root**, and a root *is* a consistent snapshot. Unaffected pages are shared. No transaction-id filtering is needed at all — but a background compaction and garbage-collection process is.

#### CONCEPT BOUNDARY — "COMMITTED" IS NOT "CONSISTENT"

**Read committed guarantees** that every individual value you read was committed at the moment you read it. **It does not guarantee** that the values were all committed *at the same moment* — and a set of individually-true numbers can be jointly impossible. That is the whole of read skew, and it is why a backup or an integrity check taken under read committed can be permanently wrong: restore it, and the disappearing money becomes real.

### Anchor sketch — the visibility ladder

Legend: bold = a named node **reversed = focus node** **Q = self-test cue**

```
READ UNCOMMITTED
     you can see writes nobody ever committed
     cheaper: no second version of the row to keep
     |
     v
READ COMMITTED       (the default in many databases)
     no DIRTY READS  -- serve the old committed value
                        while the new one is in flight
     no DIRTY WRITES -- row locks, held to commit/abort
     |
     +--> but every read happens at a DIFFERENT moment
     |
     +--> READ SKEW  $500 + $400 = $900, and $100
     |    looks gone. every number was true.
     |    the SET of them never was.
     |
     v
SNAPSHOT ISOLATION
     the whole transaction reads ONE moment: everything
     committed when it started, nothing after
     |
     +--> built on MVCC: many versions per row, each
     |    tagged with the id of the transaction that
     |    wrote it; an update is a delete plus an insert
     |
     +--> readers never block writers,
     |    writers never block readers
     |
     +--> and it is called "repeatable read" in one
          product and "serializable" in another
```

**Q** Give the two guarantees of read committed, and how each is implemented.

**Q** Why is read skew not a dirty read, and why is it intolerable for a backup?

**Q** State the four visibility rules, or at least the two-clause version of them.

## Trade-offs

| DECISION | BUYS YOU | COSTS YOU | CHOOSE IT WHEN |
|---|---|---|---|
| Read committed | No dirty reads, no dirty writes, very little overhead. Widely the default. | Read skew, lost updates, write skew and phantoms all remain possible. | Short transactions that read and write a single object, or where a momentary inconsistent view is genuinely harmless. |
| Preventing dirty reads with read locks | Simplicity — one lock mechanism for reads and writes. | One long write transaction blocks every reader of that row, so a slowdown propagates into unrelated parts of the application. | Rarely. Keeping the old committed value is the better implementation and the more common one. |
| Read uncommitted | Slightly cheaper: no need to store a second version of the row. | Dirty reads, and therefore cascading aborts. Only *reduces the probability* of lost updates. | Almost never for application data. |

| | | | |
|---|---|---|---|
| Snapshot isolation | Read skew gone. Long read-only queries — backups, analytics, integrity checks — run on a frozen, coherent view while writes continue. | Storage and garbage collection for old versions. Write skew still possible. Lost-update behaviour varies by product. | Almost always worth it over read committed, and it is the highest level some products offer. |
| Long-running read transactions under MVCC | A perfectly coherent view of an old moment, for as long as you like. | The database must retain every version that transaction might still need, so garbage collection stalls and storage grows. | Analytics and backups — but watch the age of the oldest open snapshot. |

## Say it so you understand it → say it like an expert

| SAY IT THIS WAY TO *UNDERSTAND* IT | SAY IT THIS WAY TO SOUND LIKE AN *EXPERT* |
|---|---|
| “It read half-finished data.” | “A dirty read. Read committed prevents it — and the reason it matters is cascading aborts as much as the confusing view.” |
| “The two totals didn't match.” | “Read skew. Each value was committed when we read it, but they were read at different moments, so the set was never simultaneously true. Snapshot isolation fixes it.” |
| “We'll take the backup at 3 a.m. when it's quiet.” | “Quiet doesn't help — the backup takes hours. Without a consistent snapshot the inconsistencies become permanent the moment we restore from it.” |
| “We're on repeatable read, so reads are repeatable.” | “Repeatable read means three different things across products. Which does ours implement — snapshot isolation, something weaker, or actual serializability?” |
| “Reads are slow because writes are locking them.” | “Under MVCC that shouldn't happen — readers never block writers and writers never block readers. If it is happening, we're taking read locks somewhere.” |
| “The old rows are still on disk.” | “Expected under MVCC — an update is a delete plus an insert, and garbage collection can't reclaim a version any open snapshot might still need. Check for a long-running transaction.” |

**PART 2 · QUESTION 1**

A nightly job exports the whole database to a file for regulatory reporting. It runs at read-committed isolation and takes about four hours. Auditors report that the exported ledger does not balance: a handful of transfers appear to have a debit with no matching credit, or the reverse. The team's proposed fix is to run the export at 3 a.m. “when almost nothing is happening”, and to re-run it if the totals don't match.

**(a)** Name the anomaly and explain why no value in the export was ever wrong. **(b)** Say why both parts of the proposed fix are the wrong shape of solution. **(c)** Name the isolation level that solves it, describe the mechanism that implements it, and state the operational cost the team should watch for once they switch.

**PART 2 · QUESTION 2**

Two services share a database. Service A runs short write transactions. Service B runs a five-minute read-only reconciliation query every hour. After a migration, Service A's p99 latency becomes spiky, and every spike lines up with Service B's query. Separately, the storage team reports the table's on-disk size growing steadily even though row counts are flat, and a colleague suggests that this proves the snapshot mechanism is leaking.

**(a)** Under a correct MVCC implementation, what should the relationship between B's reads and A's writes be, and what does the observed blocking tell you about how the database is configured? **(b)** Explain the disk growth without assuming a bug, and name the specific thing to look for. **(c)** The team wants B to keep its coherent view but stop affecting A. Say what you would change and what you would monitor.

## 2×2 — how many moments, and how clean

COLUMNS: CAN THE TRANSACTION SEE *UNCOMMITTED* DATA? →

| | **Yes** | **No — committed values only** |
|---|---|---|
| **MANY moments in one transaction** | **Read uncommitted**<br>Sees the latest written value even from a transaction still in flight.<br>*Move:* avoid for application data. It buys only the storage of a second row version, and it hands you dirty reads and cascading aborts in exchange. | **Read committed**<br>Every value you read was committed — and they were committed at different times.<br>*Move:* know exactly what this is worth. No dirty reads, no dirty writes. But read skew is *permitted here by design*, so anything that scans broadly — a backup, an integrity check, a report — is unsafe at this level. |
| **ONE moment** | **Incoherent**<br>A snapshot of a moment that includes writes nobody has committed is not a snapshot of any state the database was ever in.<br>*Move:* none — this cell tells you that "consistent snapshot" already implies "committed only". The two ideas are not independent. | **Snapshot isolation**<br>The whole transaction reads the database as it stood when the transaction began.<br>*Move:* take it for anything long or broad. Implemented with MVCC, so reads never take locks. Remember what it still permits: write skew always, and lost updates in some products. |

**Read it as:** the column is about *cleanliness* and the row is about *coherence*, and they are different axes — which is exactly the confusion the focus cell exists to break. Read committed is perfectly clean and completely incoherent, and people assume the first buys the second.

### CARRY-OUT RULE FROM THIS PART

**Ask how many moments your transaction is looking at.** If it reads more than one thing and the relationship between them matters, read committed is not enough, however committed each value was. Anything that scans, sums, reconciles or backs up needs a single moment.

3

PART 3 OF 4

# What they don't stop

Lost updates, write skew, phantoms — and the only three ways to buy the promise back.

### TENTH-GRADER VERSION

- Every remaining bug has the same three steps: look at something, decide based on what you saw, then write.
- By the time you write, what you saw may have changed — and your decision was based on the old version.
- If both people were changing the *same* thing, there are five different ways to stop it.
- If they were changing *different* things, almost none of those ways work, because there's no shared thing to guard.
- And if the thing they checked for was something that *didn't exist yet*, you can't even lock it — there's nothing there to lock.
- Which leaves three ways to get the real guarantee: do one at a time, lock everything, or let everyone run and cancel the ones that turned out wrong.

### THE EVERYDAY VERSION

A hospital ward needs at least one doctor on call. Two doctors are on, both feel ill, and both open the rota app at the same instant. Each one's app checks “are there two or more on call?” — yes, both times — so each one is allowed to sign off. Each writes to *their own* row. Both writes succeed. Nobody is on call. No rule was broken by either transaction on its own, no one overwrote anyone, and if the two had happened a second apart the second doctor would have been refused. That is write skew, and only the strongest isolation level catches it.

## The terms, each with its look-alike

| TERM | PLAIN DEFINITION | CONCRETE EXAMPLE | BOUNDARY — WHAT IT GETS CONFUSED WITH |
|---|---|---|---|
| **Lost update** | Two transactions do a **read-modify-write** cycle on the same object; the second write does not include the first's modification, so it is clobbered. | Two increments of a counter at 42 leave it at 43. Also: two people editing a wiki page, each sending the whole page back; adding an element to a list inside a JSON document. | vs a **dirty write**, where the first write hadn't committed. Here it had. Nothing was overwritten mid-flight — the second transaction simply read a value that was current, and wrote a value derived from it after someone else had moved on. |

| | | | |
|---|---|---|---|
| **Write skew** | Two transactions read the same objects, then update **different** objects, and the pair together violates an invariant neither broke alone. | Both on-call doctors check "≥ 2 on call", both see 2, and each takes *themselves* off call. | vs a **lost update**, which is the special case where the different transactions update the *same* object. Write skew is the generalisation. The consequence is practical: single-object atomic operations don't help, and the lost-update detection built into some snapshot-isolation implementations does not detect it. |
| **Phantom** | A write in one transaction changes the result of a **search query** in another. | Checking there is no overlapping booking for a room, then inserting one. Checking a username is free, then taking it. Checking a balance stays positive, then inserting a spend. | vs write skew on **existing rows**, where the rows the decision depended on are already there and can be locked. Here the query looks for the **absence** of matching rows, so an explicit lock has nothing to attach to. Snapshot isolation avoids phantoms in read-only queries; in read/write transactions they produce the hardest cases of write skew. |
| **Atomic write operation** | An update the database performs itself, removing the read-modify-write cycle from your code. | `UPDATE counters SET value = value + 1 WHERE key = 'foo'`. Also document-local modifications and data-structure operations in other stores. | Usually the best solution **when your change fits**. It doesn't fit arbitrary text editing. Object-relational mappers make it easy to write an unsafe read-modify-write cycle by accident instead — a bug that testing rarely finds. |
| **Explicit lock** (`SELECT … FOR UPDATE`) | The application locks the rows it is about to depend on, forcing concurrent readers of those rows to wait. | A multiplayer game where a move must satisfy rules that can't be expressed as a query: lock the piece's row, validate, update. | vs relying on the database to notice. This works, but you must reason it through — it is easy to forget a lock somewhere and reintroduce the race. Locking several objects also risks **deadlock**; databases detect that and abort one transaction, which the application must retry. |
| **Automatic lost-update detection** | Let the cycles run in parallel, detect the lost update, abort the loser and make it retry. | Several snapshot-isolation implementations do this and it composes efficiently with the snapshot machinery. | vs assuming **every** product does. At least one major engine's repeatable-read level does not detect lost updates — which is why some authors argue it doesn't provide snapshot isolation at all. The advantage of detection is that it needs no special code, so you cannot forget it; the cost is that you must handle aborts. |

| | | | |
|---|---|---|---|
| **Conditional write** (compare-and-set) | Apply the update only if the value hasn't changed since you read it; otherwise it has no effect and you retry. | `UPDATE wiki_pages SET content = 'new' WHERE id = 1234 AND content = 'old'`, or the same with a version-number column you increment — sometimes called **optimistic locking**. | vs assuming it is always safe. Check that the update took effect and retry if not. Note also a subtlety: many MVCC implementations make an exception to the visibility rules so that concurrently-written values *are* visible when evaluating the `WHERE` clause of an `UPDATE` or `DELETE`, even though they are invisible elsewhere in the snapshot. |
| **Materializing conflicts** | Creating rows whose only purpose is to be locked, so a phantom becomes an ordinary lock conflict. | A table of every room and every 15-minute slot for the next six months. A booking transaction locks the relevant slot rows first, then checks and inserts. | vs a real solution. It is hard and error-prone to work out how to materialise a given conflict, and it leaks a concurrency-control mechanism into your data model. Treat it as a **last resort** when nothing else is possible; a serializable isolation level is preferable in most cases. |
| **Serializability** | The strongest isolation level: the result is the same as if the transactions had run one at a time. | If each transaction is correct on its own, it stays correct under concurrency. All the race conditions in this booklet are prevented. | vs snapshot isolation, whatever the vendor calls it. Three implementations exist and no more: **actual serial execution**, **two-phase locking**, and **serializable snapshot isolation**. |
| **Actual serial execution** | Run one transaction at a time on a single thread. Serializable by definition, because there is no concurrency to control. | Feasible because RAM became cheap enough to hold the active dataset, and because operational transactions are short. Transactions must be submitted whole, as **stored procedures**, not statement-by-statement. | vs the interactive style. If the database waits for the application to send the next statement, single-threaded throughput is dreadful — most of the time is network round trips. Modern implementations use general-purpose languages rather than vendor-specific procedure dialects. |
| **Two-phase locking (2PL)** | Shared locks for readers, exclusive for writers; all locks held until the transaction ends. A **growing** phase acquires and a **shrinking** phase releases, and they must not overlap. | The standard way to implement serializability for around thirty years, and still the serializable level in some products. | vs **2PC**, which is a completely different thing — atomic commit across nodes, not isolation. Also vs snapshot isolation's mantra: under 2PL **writers block readers and readers block writers**, which is precisely the property MVCC was built to avoid. |

| | | | |
|---|---|---|---|
| **Predicate lock · index-range lock** | A lock on *all objects matching a search condition*, including ones that don't exist yet — which is what makes 2PL prevent phantoms. The index-range form is a cheaper approximation. | Locking "bookings for room 123 between noon and 1 p.m.". Approximated by locking the index entry for room 123, or a range of a time index. | vs a row lock, which cannot cover rows that don't exist. Predicate locks perform badly when many are active, so most systems approximate. It is **safe to over-approximate** — locking a bigger set than necessary — and the safe fallback when no suitable index exists is a shared lock on the whole table. |
| **Serializable snapshot isolation (SSI)** | Snapshot isolation plus a detector: let transactions run without blocking, and at commit time abort any whose execution wasn't serializable. | Comparatively new. Used by single-node databases, distributed databases and embedded engines. | vs 2PL, which is **pessimistic** — wait whenever something might go wrong. SSI is **optimistic** — proceed and check at the end. Optimism performs badly under high contention, because a large share of transactions abort and the retries add load to a system already near capacity. |

## The five ways to prevent a lost update — and what each assumes

1. **Atomic write operations.** Best when the change can be expressed as one. Implemented by locking the object exclusively while it is read, or by running all such operations on one thread.
2. **Explicit locks** in application code. Works, needs care, and brings deadlock risk.
3. **Automatic detection**, then abort and retry. Least error-prone, because you cannot forget it — but product-dependent.
4. **Conditional writes** (compare-and-set), including the version-number variant. The option available when there are no transactions at all.
5. **Conflict resolution after the fact**, in replicated systems. Locks and conditional writes all assume **a single up-to-date copy of the data**, which multi-leader and leaderless replication cannot provide. Instead, concurrent writes create conflicting versions — siblings — which application code or special data structures merge. Merging prevents lost updates when the updates are **commutative**: incrementing a counter and adding to a set are; a conditional write is not. And be aware that last-write-wins, the default conflict resolution in many replicated databases, is itself prone to lost updates.

## Why write skew is worse, and where the options run out

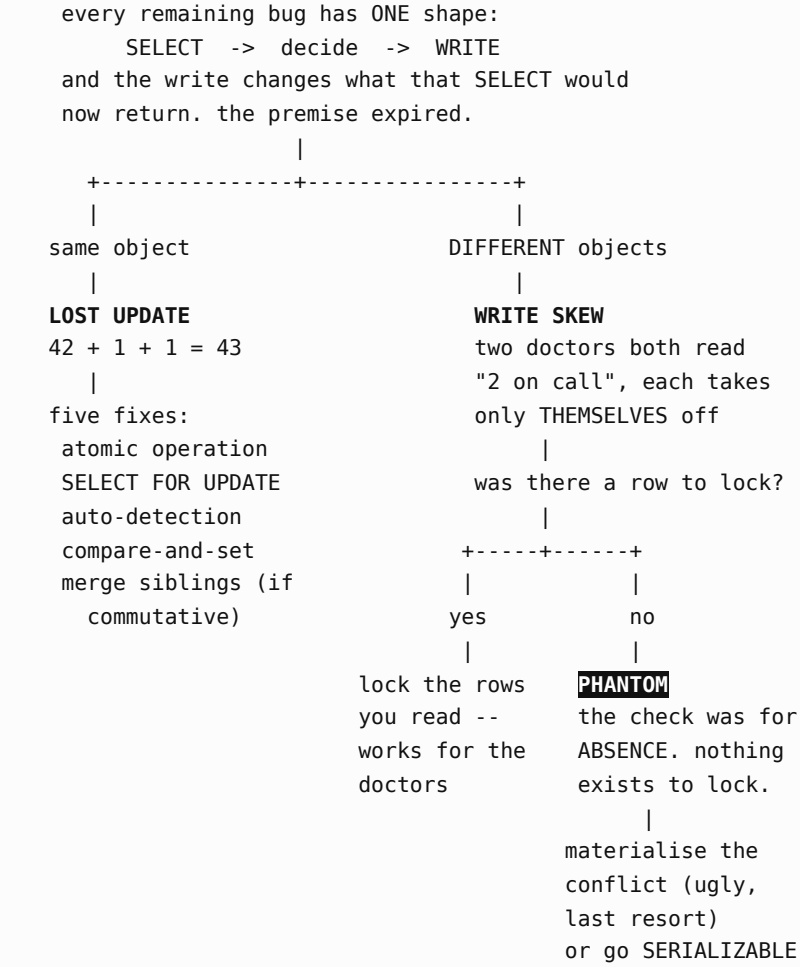
- Every example follows the same three steps. A `SELECT` checks whether a requirement holds; the application decides what to do based on the answer; it writes. **The write changes the precondition of the decision** — re-run the `SELECT` afterwards and you get a different answer. (The steps can also come in a different order: write first, then query, then decide whether to commit.)
- **Atomic single-object operations don't help**, because multiple objects are involved.
- **Automatic lost-update detection doesn't help either.** Write skew is not detected at the snapshot-isolation levels of the major products, whatever those levels are named. Preventing it automatically requires true serializable isolation.
- **Constraints usually can't express it.** "At least one doctor on call" spans multiple rows, and most databases have no built-in support for multi-object constraints — though triggers or materialized views can sometimes be pressed into service. Where a constraint *does* fit, use it: for claiming a username, a plain uniqueness constraint solves the problem outright, and the second transaction is aborted for violating it.
- **The second-best option is an explicit lock** on the rows the decision depends on — and this works for the doctors, because the row being updated in step 3 was one of the rows returned in step 1.

- **It does not work for the others.** Room booking, the username, the game position and the double-spend all check for the **absence** of rows matching a condition. If the query returns nothing, there is nothing for the lock to attach to. That is a phantom, and it is where you either materialise a conflict or go serializable.

> **CONCEPT BOUNDARY — WRITE SKEW IS NOT A LOST UPDATE, AND A PHANTOM IS NOT AN ANOMALY ON ITS OWN**
>
> **A lost update** is two transactions writing the same object. **Write skew** is two transactions reading the same objects and writing different ones — the general case, of which a lost update is the special case where the objects coincide. **A phantom** is not itself the bug; it is the *mechanism* that makes certain write skews unfixable by locking. The bug is still write skew. Saying “we have a phantom problem” is only useful if you go on to say which decision was invalidated.

## Anchor sketch — where each anomaly lives, and what stops it

**Legend: bold = a named node** **reversed = focus node** **Q = self-test cue**




**Q** Say why write skew is the general case and a lost update the special one.

**Q** Two of the five lost-update fixes fail for write skew. Which two, and why?

**Q** Which of the write-skew examples is solved by a plain uniqueness constraint, and why don't the others yield to one?

## The three roads to serializable, side by side

| APPROACH | HOW IT WORKS | WHAT IT COSTS | WHERE IT BREAKS DOWN |
|---|---|---|---|
| **Actual serial execution** | One transaction at a time on a single thread. No conflict detection needed at all. Requires transactions submitted whole as stored procedures, with the active dataset in memory. Read-only work can run outside the loop on a snapshot. | Throughput limited to one CPU core. Transactions must be short and fast, because one slow one stalls everything. Stored procedures are harder to debug, version, test and monitor than application code, and badly written ones hurt a shared database far more than a shared app server. | High write throughput. You can shard to get a thread per core, but any transaction touching several shards must run in lockstep across them, and cross-shard throughput is orders of magnitude lower and doesn't improve by adding machines. |
| **Two-phase locking** | Shared and exclusive locks on every object touched, held until the transaction ends. Predicate or index-range locks extend this to rows that don't exist yet, which is what makes it prevent phantoms. | Reduced concurrency by design: if two transactions might conflict, one waits. Unstable latency, and very bad high percentiles under contention. A transaction reading a whole table takes a shared lock on the whole table, so the database is effectively unavailable for writes to it meanwhile. Deadlocks are much more frequent than at weaker levels, and each one throws away a transaction's work. | Any contended workload, and any workload mixing long reads with writes. This is why it stopped being the default. |
| **Serializable snapshot isolation** | Snapshot isolation, plus tracking of two things: reads that ignored an uncommitted write which later committed, and writes that affect data another transaction has already read. The second acts as a **tripwire**, not a lock — it notifies rather than blocks. At commit, a transaction that acted on an expired premise is aborted. | Bookkeeping overhead, tunable by granularity: fine tracking aborts fewer transactions but costs more; coarse tracking is cheaper and aborts more than strictly necessary. And you must handle aborts and retries. | High contention, where the abort rate climbs and retries add load to an already loaded system. Long read/write transactions are likely to conflict and abort — though long read-only transactions are fine. |

### WHY SSI WAITS UNTIL COMMIT TO ABORT

When a transaction reads a value while ignoring another transaction's uncommitted write, the database could abort it there and then. It doesn't, for three reasons. The reader might turn out to be **read-only**, in which case there is no write skew to worry about and no reason to abort at all — and at read time the database doesn't yet know. The writer might still **abort**, in which case the read was never stale. And the writer might simply not have committed by the time the reader commits. Deferring the decision avoids unnecessary aborts, which is what preserves snapshot isolation's support for long-running reads.

**PART 3 · QUESTION 1**

A ticketing system holds a fixed pool of seats. To sell a seat, a transaction counts unsold seats in the section, and if the count is above zero, inserts a sale row. Under load, sections occasionally oversell by one or two. An engineer proposes locking the counted rows with `SELECT … FOR UPDATE`. Separately, the same system has a "seat notes" field that agents edit by reading the whole note, appending a line and writing it back; notes occasionally lose an agent's line.

**(a)** Name both anomalies, say which one is the general case, and explain why. **(b)** Say whether the proposed `FOR UPDATE` fix works for the overselling, and justify the answer mechanically. **(c)** Give the most appropriate fix for the lost note lines and one alternative, and say what each assumes about the data.

**PART 3 · QUESTION 2**

A team decides to move to serializable isolation. Their workload is: many short read/write transactions on a small hot set of rows, plus a few long analytical reads. They have two candidate databases — one implementing serializability with two-phase locking, one with serializable snapshot isolation — and a third proposal to run everything through single-threaded stored procedures. Their stated goals are "predictable p99 latency" and "no application changes".

**(a)** Predict the specific failure each of the three options will show on *this* workload. **(b)** Say which goal is unachievable regardless of choice, and why. **(c)** Name the property of their workload that most determines the answer, and say what you would measure before committing.

## Say it so you understand it → say it like an expert

| SAY IT THIS WAY TO *UNDERSTAND* IT | SAY IT THIS WAY TO SOUND LIKE AN *EXPERT* |
|---|---|
| "One of the two updates disappeared." | "A lost update — two read-modify-write cycles on the same object, and the second clobbered the first. Atomic operation, conditional write, or an explicit lock." |
| "Both requests passed the check and both went through." | "Write skew. They read the same rows and wrote different ones, so nothing was overwritten and nothing is detected. Only serializable prevents it automatically." |
| "We'll just lock the rows we checked." | "Only if the check was for rows that exist. If it checks for their absence, the lock has nothing to attach to — that's a phantom." |
| "Snapshot isolation detects lost updates, so we're fine." | "Some implementations do and at least one doesn't — and none of them detect write skew. Those are separate questions." |

| | |
|---|---|
| “We turned on serializable and everything got slow.” | “Which implementation? Under 2PL that's reduced concurrency and lock waits by design. Under SSI it would show up as an abort rate, not as blocking.” |
| “Optimistic concurrency control is better.” | “Better when there's spare capacity and contention is low. Under high contention the aborts and retries add load to a system already at its limit.” |
| “We'll add a table of lock rows.” | “Materialising conflicts. It works, but it's error-prone and it leaks concurrency control into the data model — last resort, after serializable has been ruled out.” |

## 2×2 — same object or not, and is there anything to lock

COLUMNS: DOES THE DECISION DEPEND ON ROWS THAT *EXIST*? →

| | **Yes — the rows are there to be read** | **No — it checks for their *absence*** |
|---|---|---|
| **Both write the SAME object** | **Lost update**<br>Two read-modify-write cycles on one row; the second clobbers the first. 42 goes to 43, not 44.<br>*Move:* five options, and you should be able to name all five. Prefer an atomic write operation; fall back to a conditional write, an explicit lock, or automatic detection. In a replicated system without a single up-to-date copy, merge siblings — and only if the operations are commutative. | **Rare in practice**<br>Two transactions can't easily contend on the same row while both having concluded that no such row exists.<br>*Move:* nothing to learn here — but notice *why* the cell is thin. “Absence” almost always means each transaction goes on to create its own new row, which puts it in the cell below. |
| **They write DIFFERENT objects** | **Write skew you can lock**<br>The doctors: both read the on-call rows, each updates their own. The rows the decision rested on are the rows being changed.<br>*Move:* serializable is the right answer; `SELECT … FOR UPDATE` on the rows read is the workable second-best, because those rows exist and can be locked. | **Write skew via a phantom**<br>The room booking, the username, the double-spend. The check is “nothing matches this condition”, and the write makes something match it.<br>*Move:* the lock has nothing to hold. A uniqueness constraint solves the narrow case where one exists. Otherwise: materialise the conflict as rows that can be locked — ugly, error-prone, last resort — or use a serializable level, which is what predicate and index-range locks were invented for. |

**Read it as:** the row tells you whether ordinary lost-update tooling applies at all; the column tells you whether locking is even physically possible. The bottom-right is the focus because it is the only cell where the usual advice — “just lock what you read” — is not merely weak but meaningless.

### CARRY-OUT RULE FROM THIS PART

**Find the decision, then ask what would change its answer.** Every anomaly here is a write that invalidates someone else's `SELECT`. If the invalidating write creates a row that didn't exist, no lock can save you and you are choosing between a constraint, a materialised conflict, and serializable isolation.

4

PART 4 OF 4

# When the transaction spans machines

Atomic commit, two-phase commit, and the in-doubt participant that holds your locks.

### TENTH-GRADER VERSION

- On one machine, “it committed” means one disk finished writing one small record. That's the whole trick.
- On several machines, you can't just tell them all to commit — some might succeed and some might fail, and you can't take back a commit once other people have read it.
- So you ask first: “can you definitely do this?” Everyone who says yes has given up the right to say no later.
- Then one machine writes down the decision, and from that moment it must be carried out, no matter how many retries it takes.
- The nasty case is the machine that says yes and then hears nothing. It can't guess. And while it waits, it's holding locks that nobody else can get past.

### THE EVERYDAY VERSION

A wedding. The officiant asks each partner separately whether they want to marry the other; each says “I do”. After both answers, the officiant pronounces them married and the fact is announced to everyone. If either says no, the ceremony is off. The key detail is what “I do” costs: before it, you are free to refuse; after it, you cannot retract it. If you faint immediately afterwards and never hear the pronouncement, you are still married — you just have to ask the officiant later which way it went.

## The terms, each with its look-alike

| TERM | PLAIN DEFINITION | CONCRETE EXAMPLE | BOUNDARY — WHAT IT GETS CONFUSED WITH |
|---|---|---|---|
| **Distributed transaction** | A transaction whose participants are more than one node. | Touching several shards of a sharded database, or a global secondary index whose entry lives on a different node from the row. | The **concurrency-control** half is broadly the same as on one node — serial execution on shards, 2PL, distributed serializability checkers for SSI. It is **atomicity** that becomes a new problem, and that is what this part is about. |
| **Atomic commitment** | Ensuring the nodes in a transaction **either all commit or all abort**, never a mixture. | Necessary because a commit cannot be retracted: once committed on one node the data is visible to other transactions, and someone may already have read it. | vs sending a commit request to every node and hoping. Some nodes may hit a constraint violation, some requests may be lost and time out, and some nodes may crash before their commit record is written and roll back on recovery. |
| **Single-node commit point** | On one machine, the transaction is committed the instant the **commit record** reaches disk, after the data. | Crash before it and the writes roll back on recovery; crash after and the transaction is committed. | The thing worth noticing: it is **one device** — the controller of one disk on one node — that makes the commit atomic. That is what disappears when the transaction spans machines. |

| | | | |
|---|---|---|---|
| **Two-phase commit (2PC)** | An algorithm for atomic commit across nodes, using a **coordinator** (transaction manager) and **participants**. Phase 1 asks everyone to prepare; phase 2 tells everyone the decision. | The coordinator is often a library inside the application process rather than a separate service. | vs **2PL**, which shares two letters and nothing else: 2PL provides serializable isolation on one node, 2PC provides atomic commit across nodes. Keep them entirely separate in your head. |
| **Prepare · the yes vote** | On receiving prepare, a participant makes sure it can commit **under all circumstances**: it writes all transaction data to disk and checks constraints and conflicts. Then it answers yes. | A crash, a power failure or a full disk is not an acceptable excuse for refusing later. | The yes vote **surrenders the right to abort** without actually committing. That is the first point of no return, and it is what one-phase commit lacks. |
| **The commit point** | The coordinator collects all votes, decides, and **writes the decision to its own log on disk**. That write is the commit point. | After it, commit or abort requests go out — and if any fails or times out the coordinator must retry **forever**. | The second point of no return. Notice what it reduces to: the commit point of a distributed transaction is an ordinary single-node atomic commit, on the coordinator. If a participant crashed after voting yes, it commits when it recovers — it cannot refuse. |
| **In doubt (uncertain)** | A participant that has voted yes and then lost contact with the coordinator. It may not abort and may not commit. | The coordinator decided to commit and reached one participant before crashing; the other has no idea which way it went. | vs a timeout being useful. A timeout does not help here: unilaterally aborting risks disagreeing with a peer that committed, and unilaterally committing risks the reverse. The participants could in principle ask each other how they voted, but that is **not part of the protocol**. The only way out is to wait for the coordinator to recover. |
| **Blocking atomic commit** | The property that the protocol can get stuck waiting for a recovery. 2PC is blocking. | An alternative three-phase protocol exists. | vs a fix. The three-phase alternative assumes a network with **bounded delay** and nodes with **bounded response times**; in real systems with unbounded delay and process pauses it cannot guarantee atomicity. The practical answer is to replace the single-node coordinator with a fault-tolerant consensus protocol. |
| **XA** | A long-standing standard for 2PC across **heterogeneous** technologies — different vendors' databases, message brokers. | Not a network protocol: a C API for talking to a transaction coordinator, with bindings in other languages. Widely implemented by relational databases and message brokers. | vs **database-internal** distributed transactions, where every participant runs the same software. Internal ones can use any protocol and optimise freely, and they often work quite well. XA is a **lowest common denominator**: it cannot detect deadlocks across systems, and it does not work with SSI, because both would need standardised cross-system protocols that don't exist. |
| **Heuristic decision** | An escape hatch letting a participant unilaterally commit or abort an in-doubt transaction without the coordinator's decision. | Offered by many XA implementations for catastrophic situations. | “Heuristic” here is a euphemism for **probably breaking atomicity**, since it violates the system of promises the protocol rests on. Not for regular use. |

| | | | |
|---|---|---|---|
| **Exactly-once semantics** | An operation takes effect once even if a crash forces a retry. | Acknowledging a message to a broker if and only if the database writes for processing it committed. | vs needing 2PC for it. You can get the same result with **only database-local transactions**, by recording processed message IDs and making the processing idempotent. That is usually the better answer. |

## Why the in-doubt state is the real problem

- A transaction holds **row-level exclusive locks** on everything it modified, to prevent dirty writes — and under 2PL it also holds shared locks on everything it read. Those locks cannot be released until it commits or aborts.
- So an in-doubt transaction **holds its locks for as long as it is in doubt**. If the coordinator takes twenty minutes to restart, the locks are held for twenty minutes. If the coordinator's log is lost, they are held until a human resolves it.
- Meanwhile no other transaction can modify those rows, and depending on the isolation level may not even read them. Large parts of the application can become unavailable.
- Rebooting the database does **not** clear it: a correct implementation must preserve an in-doubt transaction's locks across restarts, or it would risk violating atomicity.
- **Orphaned** in-doubt transactions do occur in practice — cases where the coordinator can never determine the outcome, because its log was lost or corrupted. Then an administrator must examine each transaction's participants, find out whether any has already committed or aborted, and apply the same outcome to the rest: a lot of manual work, under time pressure, during a serious outage.
- And where the coordinator is a library inside the application process, the coordinator dies when the application server dies, and its log lives on that server's local disk — which makes that disk as important as the databases themselves.

> **CONCEPT BOUNDARY — TWO VERY DIFFERENT THINGS CALLED “DISTRIBUTED TRANSACTIONS”**
>
> **Database-internal:** every participant runs the same software. Free to use a better protocol, replicate the coordinator, let the coordinator and shards talk directly, replicate the shards so a single fault doesn't force an abort, and pair atomic commit with a concurrency-control protocol that supports cross-shard deadlock detection and consistent reads. These often work well, and modern distributed databases are built on them.
> **Heterogeneous:** participants are different technologies. Constrained to the lowest common denominator, with the coordinator typically inside the application, and the application code itself is a single point of failure that replicating the coordinator does not fix.
> Criticism of “distributed transactions” usually means the second. Applying it to the first is a category error.

## Anchor sketch — the two points of no return

Legend: bold = a named node **reversed = focus node** **Q = self-test cue**

```
the problem: all commit, or all abort. never a mixture,
because a commit cannot be retracted -- someone has
already read it.
                        |
PHASE 1   coordinator -> "can you commit?"  (prepare)
          participant writes ALL its data to disk and
          checks every constraint, then votes YES
          |
          +--> POINT OF NO RETURN 1: a yes surrenders
               the right to abort, without committing.
               a crash or a full disk is no longer an
               acceptable excuse.
          |
PHASE 2   coordinator writes its DECISION to its own
          disk log
          |
          +--> POINT OF NO RETURN 2: that write IS the
               commit point -- an ordinary single-node
               commit. after it: retry forever.
          |
          v
     commit / abort  ->  every participant
                        |
and if the coordinator dies between the two?
          |
          +--> the participant is IN DOUBT. it cannot
               guess: aborting may disagree with a peer
               that committed, and committing may
               disagree with one that aborted.
          |
          +--> it waits. and it HOLDS ITS LOCKS while
               it waits -- across restarts, and until a
               human intervenes if the log is lost.
```

**Q** Name both points of no return and say what each one gives up.

**Q** Why doesn't a timeout rescue an in-doubt participant?

**Q** Why is holding locks the thing that turns a stuck transaction into an outage?

## Exactly-once, without any of this

The headline use of heterogeneous distributed transactions is committing a message acknowledgment and the database writes for processing it as one unit, so a message is effectively processed exactly once. You don't need it. Four steps, using only transactions inside the database:

1. Give every message a unique ID and keep a table of processed IDs. On receiving a message, begin a database transaction and check for its ID. **Already there?** Acknowledge to the broker and drop it.
2. Not there: insert it, process the message — including any other database writes, in the same transaction — and commit.
3. Once the commit succeeds, acknowledge the message to the broker.
4. Once acknowledged, delete the ID (in a separate transaction).

Every crash point is covered. Crash before the commit and the transaction aborts, so the broker redelivers. Crash after committing but before acknowledging and the broker redelivers, but the retry finds the ID and drops it. Crash after acknowledging but before deleting the ID and you have left a stale row, which costs a little storage and nothing else. And

if a retry arrives while the first attempt is still open, a **uniqueness constraint** on the ID table stops both from inserting it. What this really does is make the processing **idempotent**; atomicity across the broker and the database was never required.

Note the honest limit: this is not an argument that internal distributed transactions are useless. They remain valuable for the *scalability* of exactly this pattern — letting the message IDs live on one shard and the processed data on others, with atomic commit across them.

**PART 4 · QUESTION 1**

An order service writes to a relational database and publishes to a message broker from different vendors. To avoid publishing orders that were never persisted, the team wraps both in a distributed transaction using the standard cross-technology protocol, with the coordinator running as a library inside the order service. During a deploy, several order-service pods are killed mid-transaction. Within minutes, unrelated parts of the product start timing out, and the incident lasts far longer than the deploy.

**(a)** Name the state those transactions are in and explain precisely why a timeout cannot resolve it. **(b)** Explain the mechanism by which killing an order-service pod caused *unrelated* features to fail, and say why restarting the databases would not have helped. **(c)** Name the escape hatch an operator might reach for, and state exactly what it costs.

**PART 4 · QUESTION 2**

Following that incident, an architect proposes: “distributed transactions are fundamentally broken — we should ban two-phase commit across the company.” Another engineer objects that the company's new sharded database uses two-phase commit internally for every cross-shard write and has been entirely reliable. The team also still needs the guarantee that a message is processed exactly once.

**(a)** Adjudicate: state the distinction the proposed ban misses, and give three concrete things the reliable case can do that the failed one cannot. **(b)** Design the exactly-once processing without any cross-technology atomic commit, and say what each step protects against. **(c)** Say what property your design actually achieves, and name one thing internal distributed transactions would still buy you.

## Say it so you understand it → say it like an expert

| SAY IT THIS WAY TO *UNDERSTAND* IT | SAY IT THIS WAY TO SOUND LIKE AN *EXPERT* |
|---|---|
| “We'll commit on both and roll back if one fails.” | “You can't roll back a commit — once it's committed another transaction may already have read it. That's why atomic commitment needs a prepare phase.” |

| | |
|---|---|
| “It's stuck, so we'll time it out and abort.” | “It's in doubt. Aborting risks disagreeing with a participant that committed. Only the coordinator's log can decide it.” |
| “2PC and 2PL are the same family.” | “Different things entirely — 2PL is serializable isolation on one node, 2PC is atomic commit across nodes. The names are an accident.” |
| “Why is the whole app down? It was one stuck transaction.” | “Because it holds its locks while in doubt, and those survive a restart by design. Anything touching those rows blocks.” |
| “Three-phase commit fixes the blocking.” | “Only under bounded network delay and bounded response times. We have neither, so it can't guarantee atomicity here. Replicate the coordinator with consensus instead.” |
| “Distributed transactions don't work.” | “Across heterogeneous technologies they're a lowest common denominator with the application as a single point of failure. Database-internal ones are a different animal and work well.” |
| “We need 2PC for exactly-once message processing.” | “We need idempotence. A processed-message-ID table with a uniqueness constraint gets us there with database-local transactions only.” |

## 2×2 — who the participants are, and who holds the decision

COLUMNS: IS THE COORDINATOR REPLICATED AND INDEPENDENTLY REACHABLE? →

| | **No — one coordinator, often in the app process** | **Yes — replicated, with automatic failover** |
|---|---|---|
| **ALL ONE database (same software)** | **Fixable by design**<br>One coordinator, but participants that all speak the same protocol.<br>*Move:* nobody should stay here — every constraint that forces the compromise is absent. | **Database-internal, done properly**<br>Coordinator replicated by consensus with automatic failover; coordinator and shards communicate directly; shards themselves replicated so one fault doesn't force an abort; atomic commit coupled to a concurrency-control protocol that can detect cross-shard deadlocks and give consistent reads.<br>*Move:* this is what a modern sharded database does for a cross-shard write, and it is why criticism of “distributed transactions” doesn't land here. |
| **DIFFERENT technologies** | **The trap**<br>Cross-vendor atomic commit with the coordinator as a library inside the application. Kill the app process and its coordinator dies with it, leaving participants in doubt, holding locks, across restarts.<br>*Move:* avoid. If you cannot, treat the coordinator's local disk as production state — because it is — and know that the recovery path ends at a human deciding each transaction by hand. | **Better, and still limited**<br>Replicating the coordinator removes one failure mode and not the decisive one: the coordinator and participants can only communicate *through the application code*, so that code remains a single point of failure.<br>*Move:* recognise the ceiling. Cross-technology commit is a lowest common denominator — no cross-system deadlock detection, no SSI. Reach for idempotence instead. |

**Read it as:** the column is the fix everyone reaches for; the row decides the outcome. Only the top row can also close the gap between coordinator and participants — which is why the same algorithm has an excellent reputation inside a database and a poor one between products.

### CARRY-OUT RULE FROM THIS PART

**Before reaching for atomic commit across systems, ask whether idempotence would do.** Most of the time the requirement is “this must take effect exactly once”, not “these must commit together” — and a unique ID with a uniqueness constraint buys the first without any of the second's failure modes.

SECTION 6

## Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **PROMISE → DISCOUNT → DISTANCE**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

**Legend:** PROMISE — what you were sold | DISCOUNT — what you were given | DISTANCE — when it spans machines | **reversed = the one node to remember** | **Q = retrieval cue**

```
GROUP 1 - PROMISE . what a transaction is for
-----------------------------------------------------------------
   many reads and writes, ONE unit: commit or abort
        +--> A  abortability. a fault discards everything.
        |       nothing to do with concurrency.
        +--> C  YOUR invariant. only what you declared.
        +--> I  formalised as SERIALIZABLE -- and quietly
        |       discounted by almost every default
        +--> D  survives a crash. no perfect durability.
   single object -> the engine already gives you A and I
   many objects  -> a real transaction (and a secondary
                    index makes almost every write many)
                                v
GROUP 2 - DISCOUNT . what the weak levels leave out
-----------------------------------------------------------------
   READ COMMITTED  no dirty reads . no dirty writes
        lets through READ SKEW -- every value was committed,
        the SET of them never was
   SNAPSHOT ISOLATION  one moment, via MVCC
        readers never block writers, and the reverse
        lets through LOST UPDATE (sometimes) and
        WRITE SKEW (always)
   the shape of every one of them:
        SELECT -> decide -> WRITE, and the write kills
        the premise the decision rested on
        same object     -> lost update. five fixes.
        different ones  -> write skew. lock what you read.
        checked ABSENCE -> phantom. nothing exists to lock.
   three roads back: serial execution . 2PL . SSI
                                v
GROUP 3 - DISTANCE . when one disk is no longer enough
-----------------------------------------------------------------
   on one node, ONE disk writing ONE commit record makes the
   commit atomic. across nodes, that device is gone.
   2PC  prepare -> vote yes (cannot abort now)
        coordinator writes decision (cannot change now)
        commit -> retry forever
        +--> coordinator dies between them: IN DOUBT, and
        |    holding locks, across restarts
        +--> across different technologies: lowest common
             denominator, the app is the failure point
   ask first: would IDEMPOTENCE have done instead?
```

### The question per node — cover the right column

#### GROUP 1 — PROMISE

| NODE | QUESTION | ANSWER IN ONE LINE |
|---|---|---|
| **Transaction** | What is it, precisely? | Several reads and writes treated as one logical unit that either commits or aborts — so a fault means you can retry knowing nothing took effect. |
| **Atomicity** | Why is the name misleading? | It has nothing to do with concurrency. It means a fault part-way through discards every write so far — abortability. Concurrency is the I. |
| **Consistency** | Why is it the odd letter out? | The invariant is the application's. The database enforces only what you declared as a constraint; anything else is your job. |
| **Durability** | What can still go wrong after a commit? | A correlated fault taking every replica, a disk or firmware bug, misused `fsync`, or asynchronous replication losing recent writes at failover. Stack the risk reductions. |
| **Single vs multi-object** | When does one become the other without you noticing? | As soon as there is a secondary index — the index is a separate object, so the record can appear in one index and not another. |
| **Retry after abort** | Name three ways it hurts. | It may have already succeeded and the ack was lost; under contention it worsens the overload; and side effects outside the database — an email — happen again. |

### GROUP 2 — DISCOUNT

| NODE | QUESTION | ANSWER IN ONE LINE |
|---|---|---|
| **Read committed** | What are its two guarantees, and how is each built? | No dirty reads — serve the old committed value while the new one is uncommitted. No dirty writes — row locks held until commit or abort. |
| **Read skew** | Why is it not a dirty read? | Everything read was committed. They were just committed at different moments, so the set was never simultaneously true. |
| **Snapshot isolation** | What does it give and what does it still permit? | Gives one moment for the whole transaction, via MVCC. Still permits write skew always, and lost updates in some products. |
| **MVCC visibility** | When is a row visible? | If the transaction that inserted it had committed when the reader started, and it is either not marked deleted or the deleting transaction had not committed by then. |
| **Lost update** | Name the five fixes. | Atomic write operation; explicit lock; automatic detection; conditional write; merging siblings in a replicated system, if the operations commute. |
| **Reversed: write skew** | Why is this the node to remember? | It is the general case — two transactions read the same rows and write different ones. No weak level detects it, no atomic operation covers it, and only true serializability prevents it automatically. |
| **Phantom** | Why can't you lock your way out? | The decision rested on rows *not* existing, so there is nothing for the lock to attach to. Use a constraint, materialise the conflict, or go serializable. |
| **The three roads** | Name them and one failure each. | Serial execution — one core, and cross-shard throughput doesn't scale. 2PL — lock waits and terrible high percentiles. SSI — abort storms under contention. |

### GROUP 3 — DISTANCE

| NODE | QUESTION | ANSWER IN ONE LINE |
|---|---|---|
| **Atomic commitment** | Why can't you just commit on each node? | Some may hit a constraint violation, lose the request, or crash before their commit record — and a commit cannot be retracted once another transaction has read it. |
| **The prepare vote** | What does a yes cost the participant? | The right to abort. It must have written everything to disk and checked every constraint, so a crash or full disk is no longer an excuse. |
| **The commit point** | Where exactly is it? | The moment the coordinator's decision reaches its own disk log — an ordinary single-node atomic commit. After that, retry forever. |
| **In doubt** | Why can't the participant decide? | Aborting might disagree with a peer that committed; committing might disagree with one that aborted. Only the coordinator's log resolves it. |
| **Held locks** | Why does one stuck transaction become an outage? | It holds its row locks the whole time it is in doubt, preserved across restarts by design, so anything touching those rows blocks. |
| **Internal vs heterogeneous** | Why does the same algorithm have two reputations? | Internally you can replicate the coordinator, let it talk to shards directly, and pair it with real concurrency control. Across technologies you get a lowest common denominator with the application as a single point of failure. |
| **Idempotence** | What does it replace, and how? | Cross-technology atomic commit for exactly-once processing: record processed message IDs with a uniqueness constraint, and use only database-local transactions. |

## Master 2×2 — the decision card you can carry to other problems

Two counts decide which tool you need: how many **things** the rule spans, and how many **systems** those things live in. It works outside databases too.

COLUMNS: DOES THE RULE CROSS A SYSTEM BOUNDARY? →

| | No — one system owns everything | Yes — two or more systems involved |
|---|---|---|
| **The rule spans ONE thing** | **Use the built-in operation**<br>A counter, a status flag, a single record's contents. The owner can change it in one indivisible step, or refuse the change if it has moved since you looked.<br>*Move:* an atomic operation or a conditional write. No transaction, no protocol, no coordination — and most people reach straight past it for something heavier. | **Make it idempotent**<br>One effect that must happen exactly once across a boundary you don't control — a payment taken, a message processed, an email sent.<br>*Move:* give the effect a unique ID, record the IDs you have handled, enforce it with a uniqueness constraint. Retries become free and the two systems never had to agree. |
| **The rule spans MANY things** | **A transaction — and name the grade**<br>The invariant relates several things: a total and its parts, a rota and its members, a booking and a room.<br>*Move:* group them, then answer the question everyone skips — *which* guarantee are you buying? If the work reads, decides, then writes, anything below serializable will eventually let both sides through. | **Agree first, or don't try**<br>Several things, several systems, and they must move together.<br>*Move:* a prepare-then-decide protocol is the only honest way, and it works when one authority owns every participant. When they are independent, expect the stuck-in-doubt case — most such requirements are the cell above in disguise. |

**The move that matters:** people jump to the bottom-right because the requirement *sounds* like “these must happen together”. Say it out loud as an invariant first — if it is really “this must take effect exactly once”, you are in the top-right and you need an ID, not a protocol.

### THE THUMB RULE, IN ONE SENTENCE

**Name the invariant, count the things it spans and the systems it crosses — then find the moment where something is read, a decision is made, and a write follows.**

It travels. A warehouse checking stock before promising a delivery, two schedulers assigning the same room, a committee where every member individually approves an overspend that is only unaffordable in total — all the same shape: a check, a decision, and a write that invalidates the check. The fix is always one of the same four: make it one thing, make it idempotent, group it and demand the real guarantee, or accept that someone has to be asked first.

SECTION 7

# Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

## Isolation anomalies

| WHAT YOU OBSERVE | MOST LIKELY DIAGNOSIS | WHAT TO DO |
|---|---|---|
| A report's totals don't reconcile, but every individual figure checks out. | Read skew — the values were read at different moments. | Run it under snapshot isolation so the whole query sees one moment. Watch the age of the oldest open snapshot afterwards. |
| A restored backup has permanently inconsistent data. | The backup was taken without a consistent snapshot, over hours of live writes. | Snapshot isolation for the backup transaction. “Run it when it's quiet” does not help — the window is the whole backup. |
| A reader saw data that was later rolled back. | Dirty read — the isolation level is below read committed. | Move to read committed. Note the knock-on: anything that read uncommitted data has to be aborted too. |
| A sale was recorded against one buyer and the invoice against another. | Dirty write — two transactions interleaved their writes to different rows. | Read committed prevents it, using row locks held until commit or abort. Almost every implementation already does. |
| Two increments produced one increment. | Lost update — concurrent read-modify-write cycles. | Use the database's atomic operation. Failing that: a conditional write, an explicit lock, or a level that detects lost updates. |
| Two edits to the same document, one silently disappeared. | Lost update on a whole-value rewrite. | A conditional write on the previous content, or a version-number column. Check the update took effect and retry if not. |
| Both requests passed the same check and both committed. | Write skew — they read the same rows and wrote different ones. | Serializable isolation. Second-best, if the checked rows exist: lock them with `SELECT … FOR UPDATE`. |
| Two bookings for the same room and time. | Write skew via a phantom — the check was for the absence of rows. | Serializable. If unavailable, materialise the conflict as lockable slot rows, knowing it's a last resort. |
| Two accounts created with the same username. | The same phantom, in the one shape a constraint fits. | A uniqueness constraint. The second transaction aborts on violation, which is exactly what you want. |
| Reads are blocking behind writes. | Read locks are in use — either the read-committed implementation that takes them, or 2PL. | Under MVCC this shouldn't happen. Check the isolation level and the read-committed implementation setting. |
| Table size grows while row count is flat. | MVCC retaining old versions that some open snapshot might still need. | Find the long-running transaction holding the oldest snapshot. This is expected behaviour, not a leak. |
| The same isolation level behaves differently on two products. | The naming problem — “repeatable read” and “serializable” are used for different guarantees. | Check which anomalies the specific product's level permits, not what it is called. |

## Serializability and distributed commit

| WHAT YOU OBSERVE | MOST LIKELY DIAGNOSIS | WHAT TO DO |
|---|---|---|
| High percentiles collapsed after enabling serializable. | 2PL: reduced concurrency by design, plus deadlocks throwing away work. | Expect it under contention. Consider an SSI implementation, whose cost appears as aborts rather than blocking. |
| Writes to a table stop entirely while a report runs. | 2PL taking a shared lock on the whole table for a full scan. | Run long reads on a snapshot instead of inside the serializable workload. |

| | | |
|---|---|---|
| A surge of transaction aborts under load, with no blocking. | SSI plus high contention — optimistic concurrency control failing where it is weakest. | Shorten read/write transactions, reduce contention with commutative atomic operations, or leave headroom so retries don't compound. |
| One slow transaction halted all transaction processing. | Actual serial execution — a single thread, so one slow transaction stalls everything. | Keep transactions short and the working set in memory. Move long read-only work out onto a snapshot. |
| Cross-shard writes are orders of magnitude slower. | Serial execution across shards must run in lockstep, and coordination dominates. | Shard so transactions stay within one shard. Adding machines will not fix cross-shard throughput. |
| A transaction is stuck and cannot be aborted. | In doubt — it voted yes and lost the coordinator. | Recover the coordinator so it can read its log. Do not time it out: only the log knows the decision. |
| Unrelated features are timing out during an incident. | An in-doubt transaction holding row locks, blocking everything that touches those rows. | Same fix. Restarting the databases will not release them — a correct implementation preserves them across restarts. |
| The coordinator's log is lost and transactions can't be resolved. | Orphaned in-doubt transactions. | Manual resolution: for each, find whether any participant already committed or aborted, and apply the same outcome to the rest. |
| Someone proposes forcing the stuck transactions through. | A heuristic decision. | Understand what it means — probably breaking atomicity. Catastrophes only, never routine. |
| A message was processed twice after a crash. | Processing that isn't idempotent, retried after an ambiguous failure. | Record processed message IDs with a uniqueness constraint, and acknowledge only after the database transaction commits. |
| Someone proposes three-phase commit to stop the blocking. | An assumption of bounded network delay and bounded response times. | Neither holds here. Replicate the coordinator with a consensus protocol instead. |

SECTION 8

# Numbers worth remembering

| NUMBER | WHAT IT IS | WHY IT MATTERS IN AN ARGUMENT |
|---|---|---|
| **42 → 43, not 44** | Two concurrent increments of a counter. | The smallest complete demonstration of a lost update, and the reason read committed is not enough — the second write wasn't dirty, it was just stale. |
| **$500 + $400 = $900** | Read skew during a $100 transfer between two accounts holding $500 each. | Every figure was committed when read. Use it whenever someone says “but the data was committed”. |
| **2 doctors → 0 on call** | Write skew: both check “at least two on call”, both see 2, each takes themselves off. | The canonical case where nothing was overwritten, no rule was broken individually, and only serializability prevents it. |
| **20 kB** | The JSON document used to motivate single-object atomicity. | Makes concrete what the storage engine already guarantees: no unparseable fragment, no spliced old-and-new value, no partially updated read. |
| **Around 30 years** | How long two-phase locking was the only widely used way to get serializability. | Explains why weak isolation became normal: for decades the only alternative had unacceptable performance. |
| **1975 · 1983** | The first SQL database, whose transaction style is still recognisable today; and the year the ACID acronym was coined. | Fifty years of near-identical semantics is why the vocabulary is shared — and the 1983 acronym is why arguments about the letters are still arguments about definitions. |
| **1991 · 2008** | The cross-technology 2PC standard, and the first description of serializable snapshot isolation. | The gap explains the field's mood: the distributed-transaction standard is old and constrained; the good isolation algorithm is recent, which is why weak levels are still everywhere. |
| **~1,000 per second** | Reported cross-shard write throughput for a serial-execution database. | Orders of magnitude below its single-shard rate, and it cannot be increased by adding machines. The number that decides whether serial execution suits you. |
| **32-bit, ~4 billion** | Transaction ids in one major MVCC implementation, and where they overflow. | Shows that the id is a real, finite resource with a cleanup process behind it — not an abstract label. |
| **15 minutes, six months** | The granularity and horizon of a materialised-conflict table: every room × every slot, created in advance. | Makes the ugliness concrete. You are pre-creating rows that hold no data, purely to have something to lock. |
| **20 minutes** | How long locks are held if the coordinator takes 20 minutes to restart. | The clearest statement of why in-doubt is an availability problem and not just a correctness one. |
| **30% to 80%** | Share of SSDs developing at least one bad block in their first four years. | Ends “it's committed, so it's safe”. Durability is stacked risk reduction, not a guarantee. |
| **32,768 hours** | The point at which a firmware bug caused a family of drives to fail. | A correlated fault: identical drives installed together fail together, taking every replica at once. |
| **Over 20 years** | How long one well-regarded database used `fsync` incorrectly. | The most economical argument that durability primitives are hard to use correctly, even for experts. |

## Glossary — the terms you should be able to define cold

| TERM | DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR |
|---|---|
| **Transaction** | Reads and writes grouped into one unit that commits or aborts. A multi-object API call is not one. |
| **Atomicity** | A fault discards every write made so far — abortability. Not the multithreaded sense, which is isolation. |
| **Consistency (ACID)** | Your invariant, enforced only where you declared it as a constraint. One of at least five meanings of the word. |

| | |
|---|---|
| **Isolation** | Concurrent transactions don't interfere. Formalised as serializability; almost never what the default gives you. |
| **Durability** | Committed data survives a crash — by disk, log, checksum and replication. Perfect durability does not exist. |
| **Dirty read / dirty write** | Reading uncommitted data / overwriting uncommitted data. Read committed prevents both. |
| **Cascading abort** | The consequence of allowing dirty reads: anything that read data later rolled back must be aborted too. |
| **Read committed** | No dirty reads, no dirty writes. Permits read skew, lost updates, write skew and phantoms. |
| **Read uncommitted** | Prevents dirty writes only. Reduces the probability of lost updates without preventing them. |
| **Read skew (nonrepeatable read)** | Different parts of the database seen at different moments. Every value committed; the set never true at once. |
| **Snapshot isolation** | One consistent moment for the whole transaction. Not serializable, whatever the product calls it. |
| **MVCC** | Several committed versions per row, filtered by transaction id. Readers never block writers, writers never block readers. |
| **Repeatable read** | A name meaning snapshot isolation, something weaker, or serializability, depending on the product. Go and check. |
| **Lost update** | Concurrent read-modify-write cycles on the same object; the second clobbers the first. Five distinct fixes. |
| **Optimistic locking** | A conditional write using a version-number column. Not a lock at all — a retry loop. |
| **Write skew** | Reading the same objects and writing different ones. The general case of lost update; only serializability prevents it automatically. |
| **Phantom** | A write that changes another transaction's search result. Deadly when the search was for the absence of rows. |
| **Materializing conflicts** | Pre-creating rows purely to be locked. A last resort that leaks concurrency control into the data model. |
| **Serializability** | The result is as if transactions ran one at a time. Three implementations exist: serial execution, 2PL, SSI. |
| **Two-phase locking (2PL)** | Shared and exclusive locks held to transaction end; a growing phase then a shrinking phase. Writers block readers. |
| **Predicate / index-range lock** | A lock on everything matching a condition, including rows that don't exist. Over-approximating is safe; the fallback is a table lock. |
| **Serializable snapshot isolation (SSI)** | Snapshot isolation plus commit-time detection of expired premises. Optimistic; fails by aborting, not by blocking. |
| **Two-phase commit (2PC)** | Atomic commit across nodes: prepare, then decide. Unrelated to 2PL despite the name. |
| **In doubt** | Voted yes, lost the coordinator. Cannot decide alone, and holds its locks — across restarts — until resolved. |
| **Heuristic decision** | A participant resolving an in-doubt transaction unilaterally. A euphemism for probably breaking atomicity. |
| **Idempotence** | Applying an operation twice has the same effect as once. Delivers exactly-once semantics without cross-system atomic commit. |

# Answer key

### PART 1 · Q1 — THE STORE THAT IS “ATOMIC, SO WE'RE COVERED”

**(a)** They have **single-object atomicity and isolation** — no reader sees a half-written document, a crash doesn't splice old and new values, an interrupted write doesn't persist a fragment — and an **atomic conditional write**, which prevents lost updates on one object by applying the change only if the value hasn't moved since they read it. Both are genuine and both are **scoped to one object**. Neither says anything about two objects changing together, which is precisely what they need. A store offering linearizable reads and conditional writes on single objects has not given anyone transactions.

**(b)** The mismatched totals are a **multi-object atomicity failure**: the order document and the summary row are separate objects, so a fault after one write and before the other leaves them permanently disagreeing, and without atomicity nobody can tell which writes took effect. The index symptom is the same fault in a place people forget: **a secondary index is a separate database object**, so without isolation a record can appear in one index and not another because the second index update hasn't happened yet. Credit for noting that this makes almost every write multi-object whether the team intended it or not.

**(c)** First reason, not depending on the network: **retrying does not undo the write that already succeeded**. Without atomicity there is nothing to roll back, so a retry of the pair re-applies the half that worked and can double-count the summary — this is the “best effort, won't undo what it has done” posture, and error recovery is now the application's problem. Second reason: **the write may have succeeded with the acknowledgment lost**, so the retry performs it twice unless they add application-level deduplication. Also creditable: retrying is pointless after a permanent error such as a constraint violation, and retrying under contention makes an overload worse.

### PART 1 · Q2 — TWO UNSAFE STATEMENTS ABOUT ACID

**(a)** The database enforces the invariants that were **declared as constraints** — a check constraint on the balance column would genuinely prevent a negative value. It cannot enforce an invariant nobody declared, and it cannot express many complex invariants at all, particularly those spanning multiple rows. The C in ACID therefore depends on how the application uses the database and is not a property of the database alone: if the rule matters, declare it, and where it can't be declared, the application must define its transactions so they preserve it. (Extra credit for noting that the word has at least five meanings and that “ACID compliant” tells you almost nothing about which isolation level you have.)

**(b)** Two of: a **correlated fault** — a power outage, or a bug that crashes every node on the same input — losing anything held only in memory across all replicas at once; **asynchronous replication** discarding recent writes when the leader becomes unavailable; a **storage-level failure**, since drives can violate their guarantees on sudden power loss, firmware has bugs, disk data can silently corrode, and `fsync` is genuinely hard to use correctly; or the data being intact but **inaccessible** because the machine holding it is dead. The general point: there is no perfect durability, only risk-reduction techniques — disk, replication, backups — that should be used together.

**(c)** The hazard is a **side effect outside the database**. Aborting the transaction rolls back the writes but does not unsend the email, so every retry sends another. The fix is to move the send out of the transaction and make it idempotent — record the intent inside the transaction, send afterwards keyed on something unique so a repeat send can be suppressed. Credit for noting the general rule: retry only after transient errors, bound the retries with exponential backoff, and remember that a retry can also duplicate the work if the commit succeeded and the acknowledgment was lost.

## PART 2 · Q1 — THE FOUR-HOUR EXPORT THAT DOESN'T BALANCE

**(a) Read skew**, also called a nonrepeatable read. Under read committed each read returns a value that was committed at the instant it was read — so no individual value in the export was ever wrong. The problem is that the export reads different parts of the database at different moments, and a transfer that lands between two of those moments is seen from one side only: the debit is captured after it happened and the matching credit before it did, or the reverse. The set of values was never simultaneously true, even though every element of it was.

**(b)** Running at 3 a.m. treats the problem as a probability, but the exposure window is the **whole four hours** of the export, not a busy period — a single transfer anywhere in it produces the defect, and quiet hours only make it rarer and therefore harder to catch. Re-running on mismatch is worse: it is a detection mechanism dressed as a fix, and it only works for imbalances that happen to be detectable by totals. A skewed export whose figures happen to balance is still wrong, and this class of error becomes **permanent** the moment anyone restores or reports from it.

**(c) Snapshot isolation.** The mechanism is multiversion concurrency control: every transaction gets an always-increasing id, every written row is tagged with the id of the writer, an update is internally a delete plus an insert, and visibility rules hide writes from transactions that were in progress when the reader started, writes from later ids, and writes from aborted transactions. So the export reads the database exactly as it stood when it began, while writes continue unblocked — readers never block writers and writers never block readers. The operational cost to watch: the database must retain every row version the long-running export might still need, so **garbage collection stalls and storage grows** for as long as the snapshot is open. Monitor the age of the oldest open transaction.

## PART 2 · Q2 — THE RECONCILIATION QUERY THAT SHOULDN'T BLOCK ANYTHING

**(a)** Under MVCC the relationship should be **no relationship at all**: reads take no locks, so readers never block writers and writers never block readers — that is the defining performance property of snapshot isolation. Observed blocking therefore says the reads are taking locks, which means the database is not serving B from a snapshot: either the read-committed implementation in use prevents dirty reads with **read locks** rather than by keeping the old committed value, or B is running under two-phase locking, where a large scan takes a shared lock over everything it reads and writers must wait for it.

**(b)** It is the expected consequence of never updating in place. Under MVCC an update is a delete plus an insert, so modified rows accumulate versions; a version can only be garbage-collected once **no open transaction could still need it**. B's five-minute snapshot pins every version that existed when it started, so five minutes of write traffic is retained on every run, and any transaction left open longer pins proportionally more. Flat row counts with growing size is the signature, not a contradiction. Look for the **oldest open transaction** — that one number explains the retention.

**(c)** Put B on **snapshot isolation** so its coherent view costs no locks — that is exactly the workload snapshot isolation exists for, alongside backups and integrity checks. If the blocking came from a read-committed implementation using read locks, change that setting rather than weakening B's guarantee. Monitor the age of the oldest open snapshot and the growth of dead row versions, since the cost has moved from latency to storage and garbage-collection lag, and a query that hangs now costs disk instead of blocking writers.

## PART 3 · Q1 — OVERSELLING SEATS AND LOSING NOTE LINES

**(a)** The overselling is **write skew**; the vanishing note lines are a **lost update**. Write skew is the general case: two transactions read the same objects and then update *different* objects, so nothing is overwritten and no single transaction breaks the rule. A lost update is the special case where the different transactions happen to update the **same** object, so the second write clobbers the first. In the seats example each transaction counts the same rows and then inserts *its own* sale row; in the notes example both agents read and rewrite the same field.

**(b) No.** The check is a `COUNT` over unsold seats and the write is an `INSERT` of a new sale row — the transaction checks for a condition and then creates a row that changes the result of that condition. That is a **phantom**. `SELECT … FOR UPDATE` can only attach locks to rows the query returned, and here the decisive rows are the ones that *do not exist yet*, so there is nothing to lock. (Contrast the on-call doctors, where the row updated in step 3 was one of the rows returned in step 1 — there the lock does work.) The real options are serializable isolation, which is what predicate and index-range locks were invented for; or materialising the conflict as pre-created lockable rows, one per seat, which is ugly and a last resort; or, if the invariant can be expressed as one, a constraint.

**(c)** Best fix for the notes: an **atomic write operation** if appending can be expressed as one — the database performs the change without a read-modify-write cycle in application code, which is the usual best answer when it fits. It may not fit arbitrary text, so the alternative is a **conditional write** (compare-and-set) on the previous content or on a version-number column, checking afterwards that the update took effect and retrying if not. What each assumes: the atomic operation assumes the change is expressible as a database-side operation on one object; the conditional write assumes there is a **single up-to-date copy** of the value to compare against — which is exactly what multi-leader or leaderless replication cannot give you, where you would instead merge conflicting versions and can only prevent lost updates if the operations commute. Also creditable: an explicit lock, or a level that detects lost updates automatically.

## PART 3 · Q2 — CHOOSING AN IMPLEMENTATION OF SERIALIZABILITY

**(a) Two-phase locking:** the hot rows are exactly where 2PL hurts — every transaction that might conflict waits for the other, latency becomes unstable and very bad at high percentiles, and deadlocks (much more frequent at this level) throw away completed work and force retries. The long analytical reads make it worse: a transaction reading large parts of a table takes a shared lock over it, so writes to that table stop until the read commits. **SSI:** no blocking, but a small hot set means high contention, and optimistic concurrency control degrades precisely there — a large share of transactions abort at commit and the retries add load to a system already near capacity. **Serial execution:** throughput is capped at one CPU core; one slow transaction stalls every other; and the whole workload must be rewritten as stored procedures submitted in advance, since an interactive statement-at-a-time transaction would leave the single thread waiting on the network.

**(b) "No application changes" is unachievable.** Every route to serializability changes what the application must do. Under 2PL and SSI the transaction can be aborted — for deadlock or for a serialization failure — so the application must detect aborts and retry them, which is the very thing many frameworks fail to do by default. Under serial execution the change is larger still: the transaction has to be submitted whole as a stored procedure. Predictable p99 is achievable, but only by choosing the implementation whose cost shows up as aborts rather than as waiting.

**(c) Contention on the hot set** — how many concurrent transactions touch the same rows, and how long each read/write transaction stays open. That single property flips the answer: with low contention and spare capacity, optimistic concurrency control outperforms pessimistic; with high contention it collapses into abort storms while 2PL merely queues. Measure the concurrent access rate to the hot rows, the duration distribution of read/write transactions, and current headroom. Also worth measuring: whether the long reads can be moved onto a snapshot outside the serializable workload, which improves every option, and whether contention can be reduced by expressing hot updates as commutative atomic operations.

### PART 4 · Q1 — THE DEPLOY THAT TOOK OUT UNRELATED FEATURES

**(a)** Those transactions are **in doubt** (uncertain): each participant received a prepare request and voted yes, which surrendered its right to abort, and then lost the coordinator before hearing the decision. A timeout cannot resolve it because the participant has no information about the outcome — if it unilaterally aborts it may end up inconsistent with a participant that committed, and if it unilaterally commits it may end up inconsistent with one that aborted. Participants could in principle poll each other, but that is **not part of the protocol**. The only correct resolution is to wait for the coordinator to recover and read its log, which is exactly why the coordinator writes its decision to disk before sending anything.

**(b)** Because a transaction holds **row-level exclusive locks** on everything it modified (and, under two-phase locking, shared locks on everything it read) until it commits or aborts — so an in-doubt transaction holds its locks for the entire time it is in doubt. Any other transaction touching those rows blocks, and depending on the isolation level may be blocked even from reading them, so features with no connection to orders stop working. Restarting the databases does not help: a correct implementation must **preserve an in-doubt transaction's locks across restarts**, because releasing them would risk violating atomicity. The incident outlasted the deploy because the coordinator was a library *inside the order-service process* — when the pod died the coordinator died with it, and its log sits on that server's local disk, making that disk part of the durable system state.

**(c)** A **heuristic decision**: many implementations of the cross-technology standard let a participant unilaterally commit or abort an in-doubt transaction without a decision from the coordinator. The cost, stated plainly, is that “heuristic” is a euphemism for **probably breaking atomicity** — it violates the system of promises the protocol rests on, so participants can end up disagreeing permanently. It exists for catastrophes, not for routine recovery.

### PART 4 · Q2 — BANNING TWO-PHASE COMMIT

**(a)** The ban conflates two different things. **Database-internal** distributed transactions have participants all running the same software; **heterogeneous** ones span different technologies and must interoperate through a standard. Three concrete things the internal case can do: **replicate the coordinator**, with automatic failover to another coordinator node — commonly using a consensus algorithm — so a single crash doesn't leave anyone in doubt; let the **coordinator and shards communicate directly**, without going through application code, which removes the application as a single point of failure; and **couple atomic commitment to a real concurrency-control protocol**, giving cross-shard deadlock detection and consistent reads across shards. A fourth: replicate the participating shards so a fault in one doesn't force an abort. The heterogeneous case can do none of these, because it is a lowest common denominator — it cannot even detect deadlocks across systems, and it does not work with serializable snapshot isolation, since both would need standardised cross-system protocols.

**(b)** Use only database-local transactions. **1.** Give every message a unique ID and keep a table of processed IDs; on receiving a message, begin a transaction and check for the ID — if present, acknowledge and drop, which protects against a redelivery of something already done. **2.** If absent, insert it, do the processing and its database writes in the *same* transaction, and commit — so a crash before commit aborts everything and the broker redelivers safely. **3.** Acknowledge to the broker only after the commit succeeds — so a crash between commit and acknowledgment leads to a redelivery that step 1 will drop. **4.** Delete the ID afterwards, in a separate transaction — a crash here leaves a stale row that costs a little storage and nothing else. A **uniqueness constraint** on the ID table covers the remaining case, where a retry arrives while the first attempt is still open: both cannot insert the same ID.

**(c)** It achieves **idempotence** — processing the same message twice has the same effect as processing it once — and idempotence is what delivers exactly-once semantics here. Atomicity across the broker and the database was never actually required. What internal distributed transactions would still buy: **scalability of this same pattern**, letting the message-ID table live on one shard and the data updated by processing live on others, with atomic commit across those shards.

#### HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness:** did you name every element, and — for this competency especially — did you name the *boundary*? Nearly every term here has a look-alike, and an answer that gets the term right while confusing it with its neighbour is worth less than it feels. **Phrasing:** could you have said it in a design review without hand-waving? If you wrote “the isolation level was too weak” where the key says “both transactions read the same rows and wrote different ones, so nothing was overwritten and nothing was detected”, the label is there and the mechanism isn't — and the mechanism is what changes anyone's mind.

SECTION 10

## Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

| ITEM | DAY 1 | DAY 3 | DAY 7 | DAY 16 | DAY 35 | FAILS |
|---|---|---|---|---|---|---|
| **1** · Atomicity as abortability, and what it isn't | | | | | | |
| **2** · Why C is the odd letter out | | | | | | |
| **3** · Durability's four failure modes | | | | | | |
| **4** · Single-object guarantees, and where they stop | | | | | | |
| **5** · Why a secondary index makes a write multi-object | | | | | | |
| **6** · The five hazards of retrying an aborted transaction | | | | | | |
| **7** · Read committed: two guarantees, two implementations | | | | | | |
| **8** · Why read locks are the bad way to stop dirty reads | | | | | | |
| **9** · Dirty read vs dirty write vs cascading abort | | | | | | |
| **10** · Read skew, and why $900 was never wrong | | | | | | |
| **11** · The two workloads read skew makes permanent | | | | | | |
| **12** · MVCC: ids, insert/delete markers, garbage collection | | | | | | |
| **13** · The visibility rules, in the two-clause form | | | | | | |
| **14** · Readers never block writers — and what it implies | | | | | | |
| **15** · Why “repeatable read” is not an answer | | | | | | |
| **16** · Lost update: all five preventions | | | | | | |
| **17** · What the fifth prevention assumes that the others don't | | | | | | |
| **18** · Write skew as the general case | | | | | | |
| **19** · Phantoms, and why FOR UPDATE has nothing to hold | | | | | | |
| **20** · Materialising conflicts, and why it's a last resort | | | | | | |
| **21** · Serial execution: what it needs, where it stops | | | | | | |
| **22** · 2PL: growing/shrinking, predicate and index-range locks | | | | | | |
| **23** · SSI: the two detections, and why it waits to abort | | | | | | |
| **24** · Pessimistic vs optimistic, and when each wins | | | | | | |
| **25** · 2PC: the two points of no return | | | | | | |
| **26** · In doubt: why no timeout helps, and why locks are the problem | | | | | | |

| | | | | | | |
|---|---|---|---|---|---|---|
| **27** · Internal vs heterogeneous distributed transactions | | | | | | |
| **28** · Exactly-once by idempotence, all four steps | | | | | | |
| **29** · Master matrix — things spanned × systems crossed | | | | | | |

## Final quiz — no notes, twenty minutes, write the answers

1. Explain why ACID atomicity would have been better named abortability, and say which letter covers a concurrent client seeing your half-finished work.

2. Give the two guarantees of read committed and how each is implemented — including the implementation that is possible but works badly, and exactly why it does.

3. Explain read skew with a concrete example, say why nothing that was read was uncommitted, and name two workloads for which it is intolerable.

4. Describe how MVCC gives a transaction a consistent snapshot: what is tagged on each row, what an update really is, and the two-clause rule for whether a row is visible.

5. Name all five ways to prevent a lost update, and say what the fifth assumes that the other four do not.

6. Define write skew, say why it is the general case of the lost update, and name two lost-update preventions that do not work against it.

7. Explain what a phantom is and why an explicit lock on the rows you read cannot help, then give the remaining options and say which is preferred.

8. Name the three implementations of serializability and give the specific workload that defeats each one.

9. State the two points of no return in two-phase commit, say what each one surrenders, and explain why a participant in doubt may not decide for itself.

10. Design exactly-once message processing without atomic commit across the broker and the database, and say what property it actually achieves.